Fuzz Testing

Hack, Art, Science

Mengfei Ren

CSE4380/5380 Information Security I

THE UNIVERSITY OF TEXAS
AT ARLINGTON

Software Testing

Why we need to test our software?

UTA

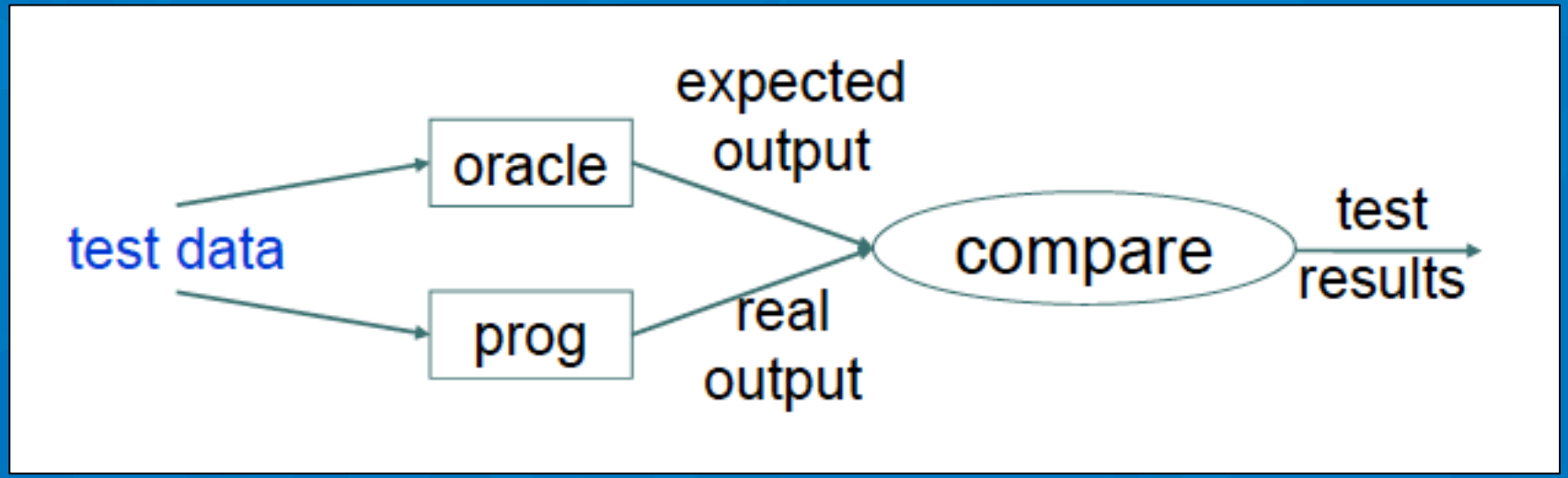
“Software testing is a process, to evaluate the functionality of a software application with an intent to find whether the developed software met the specified requirements or not and to identify the defects to ensure that the product is defect free in order to produce the quality product.”

-- Wikipedia

Sources of Bugs

- Documentation
- Design
- Implementation
- System Configuration

Testing Process



Test Data Selection

- Testing is w.r.t. a finite test set
  - Exhaustive testing is usually not possible
  - E.g, a function takes 3 integer inputs, each ranging over 1 to 1000
- **Question: How do you design the test set?**

Testing Approach

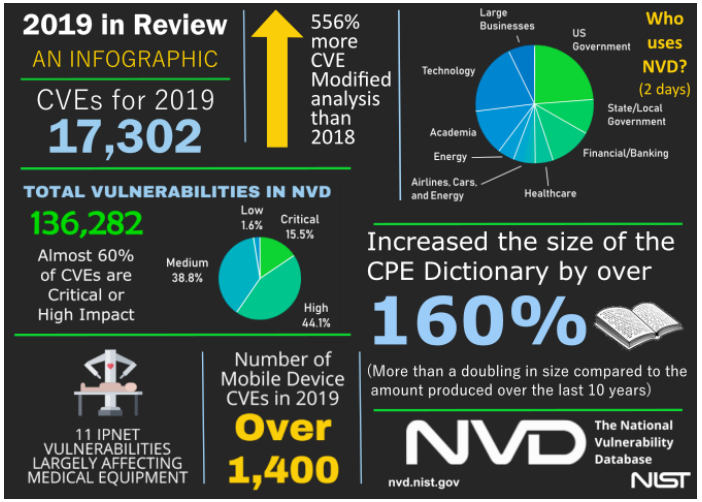
- **White-box testing**
  - Has knowledge of the system under test (e.g., program structure)
  - Tests internal structures or workings of an application
- **Black-box testing**
  - Test functionality without any knowledge of internal implementation
  - Advantage:

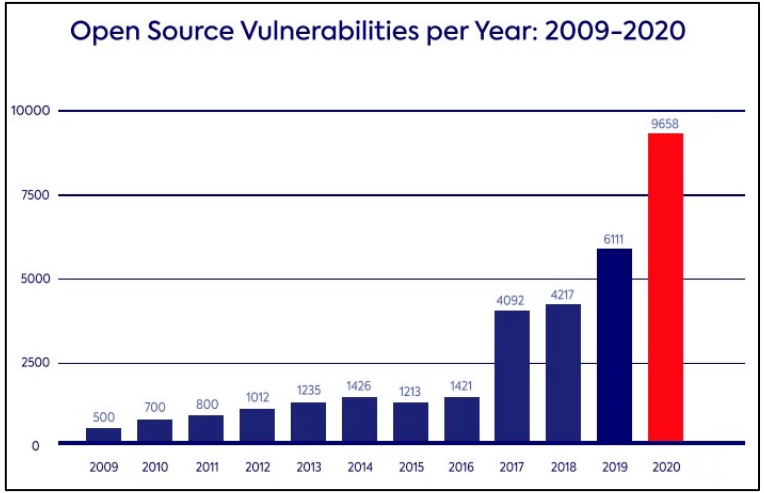
    Test cases are not biased toward an implementation

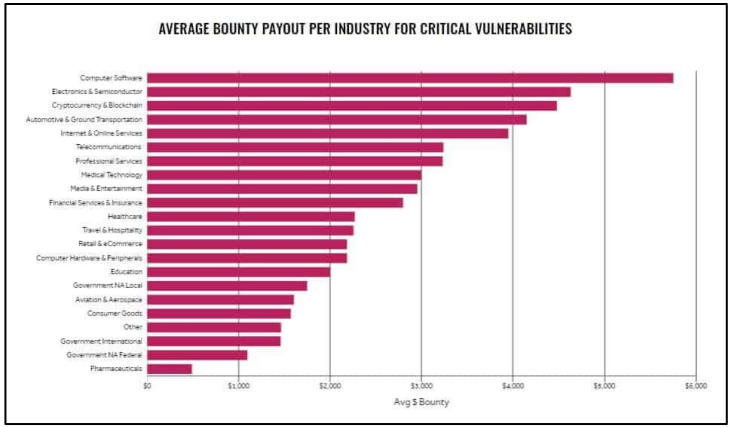
Software Security

Security vulnerability detection brings huge money

UTA



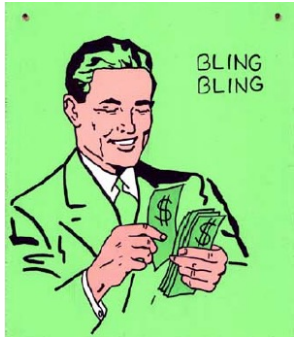




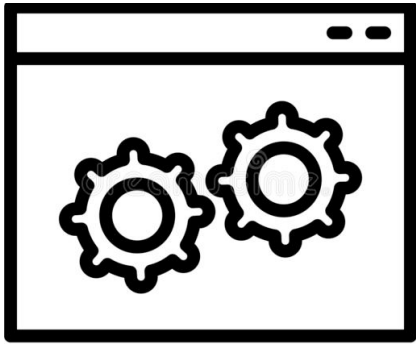
A large number of software vulnerabilities are reported every year!

Vulnerability Detection Today

- Security bugs can bring $500-$100,000 on the open market
- Good bug finders make $180-$250/hr consulting
- Few companies can find good people, many don’t even realize this is possible.
- Still largely a black art



How to generate a large set of pictures



Picture Parser

?

How secure is your JPEG parser?

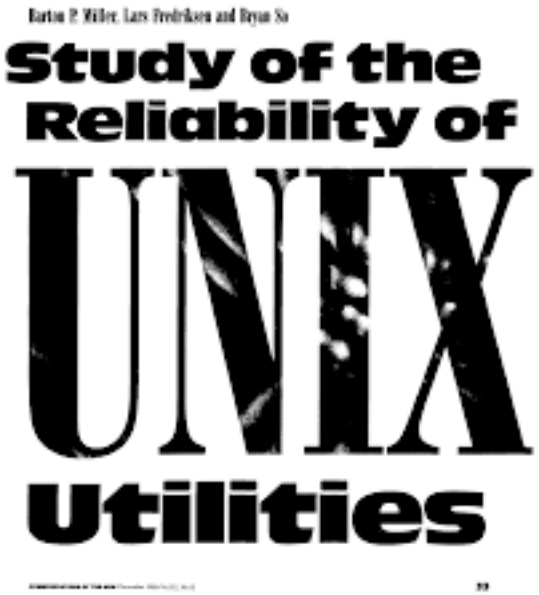
Designing A Good Test Suite is Hard

- Figuring out a good test set is a major task
- 100% coverage almost never achieved in practice
- **One idea:**
  - Randomly generate test data (fuzzing)

Fuzz Testing

To find program inputs that reveal a bug

Fuzz Testing


Barton P. Miller, Lars Fredriksen and Bryan So

Study of the Reliability of UNIX Utilities

Communications of the ACM (1990)

> On a dark and stormy night one of the authors was logged on to his workstation on a dial-up line from home and the rain had affected the phone lines; there were frequent spurious characters on the line. The author had to race to see if he could type a sensible sequence of characters before the noise scrambled the command. This line noise was not surprising; but we were surprised that these spurious characters were causing programs to crash.

Definition

- Fuzz - a blurred image or area
- Fuzz Testing: Run program on many **random**, **abnormal** inputs and look for bad behavior in the response.
  - Bad behaviors: crashes, hangs, ...
- Becomes a mainstream practice in assessing software security since 2000.

Bugs & CWEs found through Code Intelligence fuzzing in C/C++

CWE-119	Improper Restriction of Operations within the Bounds of a Memory Buffer
CWE-823	Use of Out-of-range Pointer Offset
CWE-786	Access of Memory Location Before Start of Buffer
CWE-680	Integer Overflow to Buffer Overflow
CWE-466	Return of Pointer Value Outside of Expected Range
CWE-787	Out-of-bounds Write
CWE-125	Out-of-bounds Read
CWE-129	Improper Validation of Array Index
CWE-131	Incorrect Calculation of Buffer Size
CWE-193	Off-by-one Error
CWE-195	Signed to Unsigned Conversion Error
CWE-839	Numeric Range Comparison Without Minimum Check
CWE-843	Access of Resource Using Incompatible Type ('Type Confusion')
CWE-1257	Improper Access Control Applied to Mirrored or Aliased Memory Regions
CWE-1260	Improper Handling of Overlap Between Protected Memory Ranges
CWE-190	Integer Overflow or Wraparound
CWE-20	Improper Input Validation
CWE-415	Double Free
CWE-416	Use After Free
CWE-476	NULL Pointer Dereference
CWE-590	Free of Memory not on the Heap
CWE-362	Concurrent Execution using Shared Resource with Improper Synchronization
CWE-364	Signal Handler Race Condition
CWE-366	Race Condition within a Thread
CWE-367	Time-of-check Time-of-use (TOCTOU) Race Condition
CWE-368	Context Switching Race Condition
CWE-421	Race Condition During Access to Alternate Channel
CWE-1223	Context Switching Race Condition
CWE-662	Improper Synchronization
CWE-758	Reliance on Undefined, Unspecified, or Implementation-Defined Behavior
CWE-562	Return of Stack Variable Address
CWE-587	Assignment of a Fixed Address to a Pointer
CWE-588	Attempt to Access Child of a Non-structure Pointer
CWE-1102	Reliance on Machine-Dependent Data Representation
CWE-1103	Use of Platform-Dependent Third Party Components
CWE-1105	Insufficient Encapsulation of Machine-Dependent Functionality

© 2021 Code Intelligence. All rights reserved.

List of bugs and CWEs that CI Fuzz found in C/C++ software (click to enlarge)

CLOUDY, WITH A CHANCE OF EXPLOITS —

Microsoft launches "fuzzing-as-a-service" to help developers find security bugs

Project Springfield, Microsoft's "million-dollar bug detector" now available in cloud.

SEAN GALLAGHER - 9/27/2016, 10:21 AM

Chromium Blog

News and developments from the open source browser project

Fuzzing for Security

Thursday, April 26, 2012

Web browsers are big, complicated pieces of software that are extremely difficult to secure. In the case of Chrome, it's an even more interesting challenge as we contend with a codebase that evolves at a blisteringly fast pace. All of this means that we need to move very quickly to keep up, and one of the ways we do so is with a scaled out fuzzing infrastructure.

Chrome's fuzzing infrastructure (affectionately named "ClusterFuzz") is built on top of a cluster of several hundred virtual machines running approximately six-thousand simultaneous Chrome instances. ClusterFuzz automatically grabs the most current Chrome LKGR (Last Known Good Revision), and hammers away at it to the tune of around fifty-million test cases a day. That capacity has roughly quadrupled since the system's inception, and we plan to quadruple it again over the next few weeks.

The remotely exploitable flaw in Log4j – the widely deployed Java error logging library -- is being attacked by multiple actors and likely will remain so for many more months as open-source projects, product vendors, and end-user organisations patch affected systems.

Google is now adding OSS-Fuzz to the pool of answers to the internet-wide Log4j flaw, also known as Log4Shell. The bug is tracked as CVE 2021-44228 and was partially fixed in Apache Foundation's release of Log4j version 2.15.0 last week.

OSS-Fuzz is Google's free service for fuzzing open-source software projects and is currently used by over 500 critical projects. Fuzzing involves throwing random code at software to produce an error, like a crash, and uncover potential security flaws.

Regression Testing vs. Fuzzing

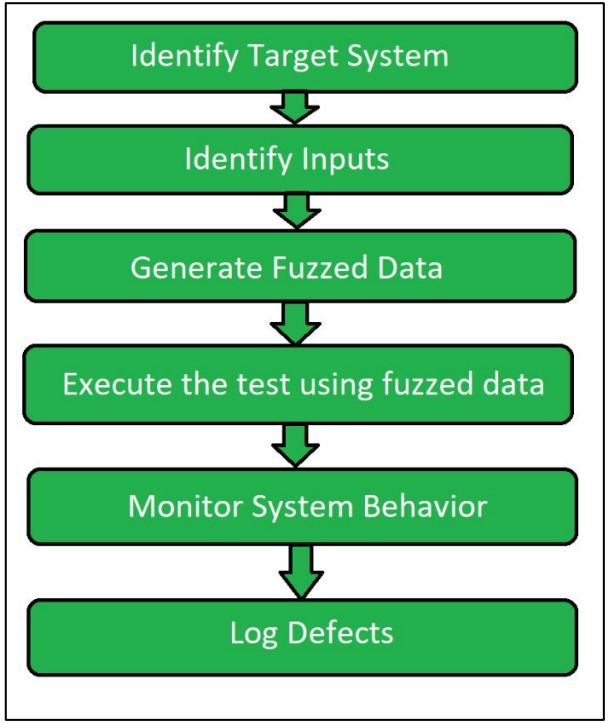
- **Regression**: Run program on normal inputs, and look for badness.
  - Prevent normal users from encountering errors.
- **Fuzzing**: Run program on many abnormal inputs, and look for badness.
  - Prevent attackers from encountering exploitable errors.

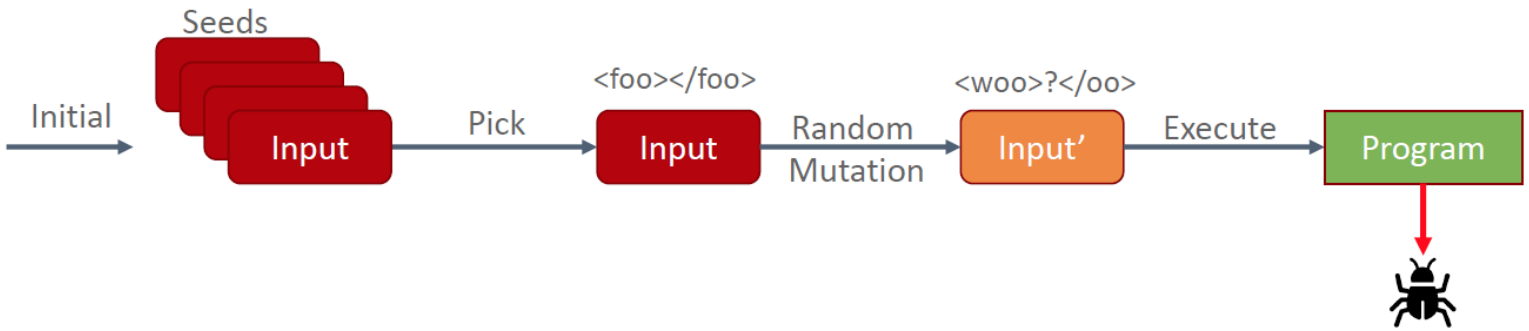
UTA

Fuzz Testing

Advantages

- No knowledge of target program required
- No source codes required
- Generate random and weird inputs for testing
- Less time consuming





Valid Seed Input (build.xml)

```
<project default="dist">
 <target name="init">
  <mkdir dir="${build}"/>
 </target>
 …
```

New Input (Mutated from Seed)

```
<project default="dist">
 <taWget name="init">
  <madir dir="2{build}"/@
 </tar?get>
 …
```

Detection of Fuzzing

What to find?

- Memory corruption in native code
- Unhandled exceptions in managed code
- Injection in web applications

What not to find?

- Hard to find logical flaws
- Hard to find design flaws

UTA

American Fuzzy Lop

Most Popular Fuzzing Tool

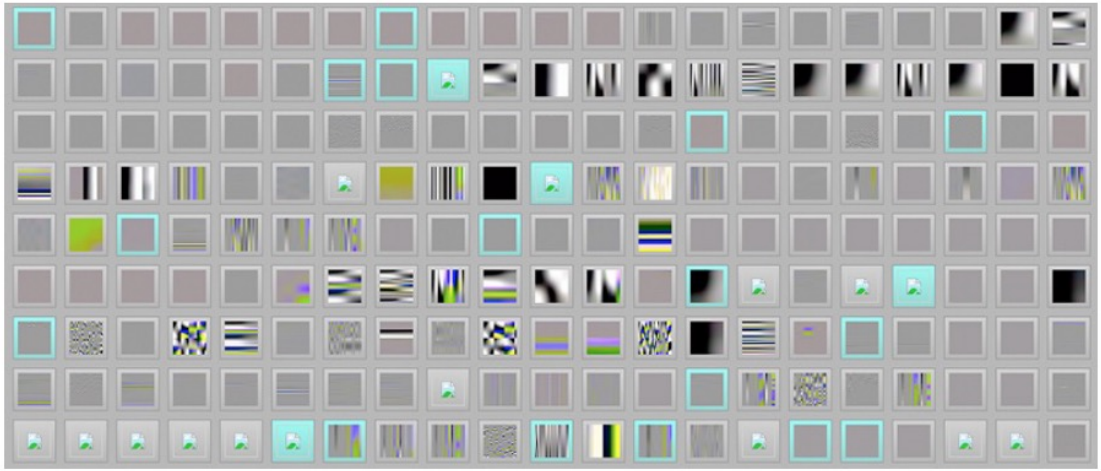
UTA

November 07, 2014

Pulling JPEGs out of thin air

This is an interesting demonstration of the capabilities of afl; I was actually pretty surprised that it worked!

```
$ mkdir in_dir
$ echo 'hello' >in_dir/hello
$ ./afl-fuzz -i in_dir -o out_dir ./jpeg-9a/djpeg
```



The bug-o-rama trophy case

http://lcamtuf.coredump.cx/afl/

IJG jpeg 1	libjpeg-turbo 1 2	libpng 1
libtiff 1 2 3 4 5	mozjpeg 1	PHP 1 2 3 4 5 6 7 8
Mozilla Firefox 1 2 3 4	Internet Explorer 1 2 3 4	Apple Safari 1
Adobe Flash / PCRE 1 2 3 4 5 6 7	sqlite 1 2 3 4...	OpenSSL 1 2 3 4 5 6 7
LibreOffice 1 2 3 4	poppler 1 2...	freetype 1 2
GnuTLS 1	GnuPG 1 2 3 4	OpenSSH 1 2 3 4 5
PuTTY 1 2	ntpd 1 2	nginx 1 2 3
bash (post-Shellshock) 1 2	tcpdump 1 2 3 4 5 6 7 8 9	JavaScriptCore 1 2 3 4
pdfium 1 2	ffmpeg 1 2 3 4 5	libmatroska 1
libarchive 1 2 3 4 5 6 ...	wireshark 1 2 3	ImageMagick 1 2 3 4 5 6 7 8 9 ...
BIND 1 2 3 ...	QEMU 1 2	lcms 1

```
                    american fuzzy lop 2.36b (                )

 process timing                                                      overall results
        run time : 0 days, 0 hrs, 5 min, 20 sec                       cycles done : 0
   last new path : 0 days, 0 hrs, 0 min, 9 sec                        total paths : 241
 last uniq crash : 0 days, 0 hrs, 0 min, 49 sec                      uniq crashes : 14
  last uniq hang : 0 days, 0 hrs, 0 min, 19 sec                        uniq hangs : 22
 cycle progress                                  map coverage
  now processing : 121 (50.21%)                    map density : 0.23% / 0.87%
 paths timed out : 0 (0.00%)                    count coverage : 2.34 bits/tuple
 stage progress                                  findings in depth
  now trying : interest 32/8                    favored paths : 51 (21.16%)
 stage execs : 3550/8883 (39.96%)                new edges on : 75 (31.12%)
 total execs : 777k                             total crashes : 140 (14 unique)
  exec speed : 3560/sec                           total hangs : 400 (22 unique)
 fuzzing strategy yields                         path geometry
   bit flips : 91/30.7k, 15/30.7k, 6/30.6k          levels : 3
  byte flips : 1/3838, 1/3542, 2/3510              pending : 217
 arithmetics : 42/198k, 3/71.9k, 0/32.0k          pend fav : 38
  known ints : 3/19.1k, 7/84.4k, 22/132k          own finds : 239
  dictionary : 0/0, 0/0, 5/23.3k                   imported : n/a
       havoc : 55/106k, 0/0                       stability : 100.00%
        trim : 22.95%/1711, 7.22%

                                                          [cpu:301%]
```

Fuzzing is commonly used as a shorthand for security testing because the vast majority of its applications is for finding security vulnerabilities.

UTA

Thank You!

Any questions?

You can find me at mengfei.ren@mavs.uta.edu